

Node-level performance of a CVFEM Navier–Stokes multigrid solver on an NVIDIA GH200 node

SFEM / CVFEM spike

September 20, 2026

Abstract

We report node-level performance of the semi-structured CVFEM Navier–Stokes solver in SFEM on a single Alps GH200 node (four sockets, 72 Neoverse V2 cores each, 288 cores, ≈ 870 GB). At a fixed core count the rank $\times$ thread decomposition is swept from 1×288 to 72×4 ; the optimum is 4×72 , one rank per socket, at both problem sizes tested. The decomposition is shown to be iteration-count invariant, so the wall-clock comparisons are between runs performing identical work. The dominant cost at every shape is the coarsest-level solve, a dense LU that each rank stores in full and assembles through an n^2 allreduce; it is a fixed ≈ 5.4 s floor that does not amortise with problem size and that bounds the attainable macro-element resolution. We also quantify the cost of a defect in the coarse hierarchy’s mesh packing, which silently tripled the iteration count at the shipped default settings.

1 Platform and configuration

All measurements are taken on one Alps GH200 node: four sockets of 72 Neoverse V2 cores, 288 cores in total, 36 NUMA domains, and roughly 870 GB of memory. The toolchain is the `prgenv-gnu/24.11:v2` user environment with `cray-mpich` under `srun`; threads are bound with `OMP_PROC_BIND=true` throughout.

The test problem is the lid-driven cavity discretised with the semi-structured CVFEM Navier–Stokes operator. Two sizes are used:

case	macro elements	nodes	dof	MG levels
$N = 16$, refine 4	4096	274 625	1 098 500	3
$N = 16$, refine 8	4096	2 146 689	8 586 756	4

The solver is FGMRES preconditioned by geometric multigrid with a Vanka smoother and Galerkin-assembled coarse operators (`SFEM_GMG_GALERKIN=2`); the coarsest level is solved by a dense LU. All runs perform one Newton step with a linear tolerance of 1×10^{-4} , which is the regime the solver is used in; reported iteration counts are therefore the linear iterations of that step.

2 Rank $\times$ thread decomposition at fixed core count

Table 1 and Table 2 sweep the decomposition at a constant 288 cores. In both, the linear iteration count is identical at every shape, so the wall-clock figures compare runs performing the same work.

The optimum is 4×72 at both sizes: one MPI rank per socket, with each thread team confined to one socket’s memory. The curve is monotone on both sides of it, and the two directions fail for different reasons.

Table 1: Decomposition sweep at 1 098 500 dof (three levels). Linear iterations are 7 at every shape.

shape (ranks $\times$ threads)	total (s)	coarse factorisation (s)	solve (s)
4×72	9.31	5.351	1.2901
8×36	11.59	7.824	1.3259
12×24	14.66	10.864	1.3852
18×16	20.24	16.214	1.5299
36×8	28.58	24.602	1.5454
72×4	49.99	45.778	1.7269

Table 2: Decomposition sweep at 8 586 756 dof (four levels), $\approx 29\,800$ dof per core. Linear iterations are 5 at every shape. The three fat shapes are from a separate job that repeats 4×72 ; the two 4×72 measurements, 11.44 s and 11.07 s, bound the run-to-run variation at about 3 %.

shape (ranks $\times$ threads)	total (s)	coarse factorisation (s)	solve (s)
1×288	21.50	10.864	5.5725
2×144	16.49	6.117	4.2616
4×72	11.07	5.322	1.6595
4×72	11.44	5.414	1.6693
8×36	12.97	7.779	1.7866
18×16	22.61	16.945	2.4573
72×4	50.74	45.385	2.4001

Towards lean ranks, the coarse factorisation dominates: it grows by $8.6\times$ from 4×72 to 72×4 while the solve phase moves only 34 %. Towards fat ranks, the failure is NUMA. At 1×288 a single OpenMP team spans all four sockets: the solve phase degrades by $3.4\times$ (1.66 s to 5.57 s), and the factorisation itself is *worse* than at four ranks (10.86 s against 5.32 s), because one dense `getrf` spread across four sockets is slower than four independent factorisations each inside a socket. The elimination of MPI replication and of the n^2 allreduce at one rank does not compensate.

At saturation the baseline wins on *both* components: even setting the coarse factorisation aside, the solve phase alone favours 4×72 by roughly $1.5\times$ over the lean shapes.

3 Where the time is spent

The phase breakdown at 4×72 and 8 586 756 dof attributes 5.414 s of 11.44 s — 47 % — to the coarsest-level factorisation; at 72×4 the same phase is 89 % of the run. Raising the problem size eightfold, from 1 098 500 to 8 586 756 dof, left that cost essentially unchanged (5.351 s against 5.414 s at the baseline shape), because the coarsest level is fixed by the macro mesh (17^3 nodes, 19 652 dof) and is untouched by refinement. It is therefore a fixed floor, not an amortisable overhead.

Table 3 separates the two components of that phase on a 740 772 dof case with a 13 500-dof coarsest level, in pure MPI.

The two costs have opposite character. The factorisation is replicated, so its cost per rank is flat and no parallelism is recovered from it. The assembly allreduce grows with rank count, from 5.2 s at 128 ranks to 54.9 s at 288. The solve phase, by contrast, scales well: 12.57 s to 1.35 s from 1 to 128 ranks, a factor of 9.3.

Resolution ceiling. The dense coarse solve costs $\mathcal{O}(n^3)$ in time and $\mathcal{O}(n^2)$ in memory, where n is the coarse dof count and scales as $(N + 1)^3$ in the macro resolution N — so roughly N^9 .

Table 3: Coarse-solve components at 740 772 dof, pure MPI (one thread per rank). `getrf` is the per-rank dense factorisation; the remainder of the phase is dominated by the n^2 allreduce that assembles the coarse matrix on every rank.

ranks	<code>getrf</code> (s)	rest of phase (s)	solve (s)
1	45.4	0.1	12.57
8	44.9	1.9	2.97
32	47.4	1.3	1.64
128	47.4	5.2	1.35
288	54.9	54.9	3.41

Extrapolating from the measured point at $N = 16$ (19 652 dof, 3.09 GB per rank, 5.4 s), $N = 24$ would need ≈ 31 GB per rank and minutes of factorisation, and $N = 32$ about 165 GB per rank. This was observed directly: at 288 ranks and a 19 652-dof coarse level the replicated factor required 288×3.09 GB ≈ 890 GB and the run was terminated by the out-of-memory killer. *The coarse solver, not the domain decomposition, is what bounds the attainable mesh resolution.*

4 Iteration-count invariance

The wall-clock comparisons above are only meaningful if the decompositions perform identical work. Table 4 confirms this over the full range of rank counts.

Table 4: Linear iterations against rank count, pure MPI. Block-Jacobi is the structural control: it inverts one 4×4 block per node, so its owned output depends only on owned input and it cannot be partition dependent. The four-level and three-level entries at 1 098 500 dof are not a duplication: $N = 8$ at refine 8 and $N = 16$ at refine 4 coincide in degree-of-freedom count while differing in hierarchy depth.

case	smoother	linear iterations
1 098 500 dof, 4 levels	Vanka	6 at 1, 8, 32, 64, 128, 288 ranks
740 772 dof, 3 levels	Vanka	7 at 1, 8, 32, 128, 288 ranks
1 098 500 dof, 3 levels	Vanka	7 at 1, 8, 32, 128 ranks
1 098 500 dof, 3 levels	block-Jacobi	10 at 1, 8, 32, 128 ranks
143 748 dof, 3 levels	block-Jacobi	11 at 1, 8, 32, 64 ranks

5 A packing defect in the coarse hierarchy

The coarse levels of the hierarchy were packed for SIMD execution, and packing rennumbers a mesh’s nodes in place. The hierarchy’s transfer operators, however, do not address the coarse vector through the coarse mesh at all: both `sshex8_hierarchical_prolongation` and `sshex8_hierarchical_restriction` take only the *fine* element table and index coarse degrees of freedom by macro-corner identifier. Renumbering the coarse mesh therefore desynchronised the coarse operator from the transfers that feed it. The permutation is the identity when a level fits in a single pack, which is why the defect was invisible below 2048 macro elements — the shipped default pack size.

Table 5 gives the cost at 1 098 500 dof. Two coarse-operator modes are shown: Galerkin assembly, the default, and rediscretisation.

At the shipped default the Galerkin path required three times the iterations and 55 % more wall clock; the rediscretisation path failed to converge at all. The repair is to decline packing

Table 5: Effect of the coarse-packing defect and its repair at 1 098 500 dof, serial, 72 threads. Pack size 2048 is the shipped default and yields two packs at this mesh size; 4096 yields one pack and 0 disables packing. “cap” denotes a run that reached the 1000-iteration limit without converging, so both its iteration count and its time are lower bounds.

coarse operator	pack size	before		after	
		iters	time (s)	iters	time (s)
Galerkin	0 (off)	7	5.80	7	5.79
Galerkin	2048	22	8.97	7	5.82
Galerkin	4096	7	5.65	7	5.87
rediscretisation	0 (off)	17	13.03	17	12.85
rediscretisation	2048	cap	222.65	17	12.97
rediscretisation	4096	17	30.79	17	12.98

on derefined levels, whose numbering is load-bearing for the transfers, while leaving the fine level’s packing — the performance-critical one — untouched. The unpacked configurations are unchanged to within run-to-run variation, confirming that the change does not perturb the path it does not touch.

6 Limitations

- **Communication overlap is not exercised.** The library’s default matrix-free path overlaps the halo exchange with computation over element subranges. The CVFEM operator does not implement a scoped apply and aborts on that path, so every measurement here uses the blocking `SFEM_PARALLEL_MF_LEGACY=1` path. The cost of the communication thread at each decomposition shape therefore remains unmeasured.
- **One Newton step.** Iteration counts and timings are for a single Newton step at a linear tolerance of 1×10^{-4} . A full nonlinear continuation has not been swept across decompositions.
- **One coarse-operator mode for the sweeps.** The decomposition sweeps use Galerkin-assembled coarse operators. Substituting an iterative (BiCGStab) coarse solve removes the factorisation cost but destroys iteration-count invariance: it neither converged to its 1×10^{-8} tolerance nor exhausted its iteration budget, stalling inconsistently and yielding outer counts that varied with decomposition and between repeat runs of the same configuration. A preconditioned or genuinely distributed direct coarse solve, rather than this substitution, is the route to lifting the ceiling in Section 3.
- **Single node.** No multi-node results are reported.